Experiment 8

WAP for division of two numbers where denominator is 0. This will throw an exception. Now handle the exception using try and except.

Theory: -

Exception handling in Python is used to manage errors that occur during program execution. It uses try, except, else, and finally blocks to handle exceptions. It helps prevent program crashes and ensures smooth execution.

Code: -

```
# WAP for division of two numbers where denominator is 0. This will throw an exception. Now handle the exception using try and except.


try:
    num1 = float(input('first number: '))
    num2 = float(input('second number: '))
    print(num1/num2)
except ZeroDivisionError:
    print('num2 is 0')
except ValueError:
    print('num1 or num2 is not int')
else:
    print('Successful')
finally:
    print('Done')
```

Output 1: -

first number: 12

second number: 0

num2 is 0

Done

Output 2: -

first number: v

num1 or num2 is not int

Done

Output 3: -

first number: 12

second number: 5

2.4

Successful

Done

Experiment 9

WAP in Python that can create your own exceptions.

Theory: -

Python allows users to create their own exceptions according to their requirements. Custom exceptions are created by defining a class that inherits from the Exception class. The raise keyword is used to generate a custom exception. Custom exceptions help handle specific errors and make programs easier to understand.

Code: -

```
class MyError(Exception):
    pass


try:
    age = int(input("Enter  your age: "))


    if age < 18:
        raise MyError("Age  must  be 18 or above")
    else:
        print("You  are eligible.")
except MyError  as e:
    print("Error:",  e)
```

Output 1: -

```
Enter your age: 15
Error: Age must be 18 or above
```

Output 2: -

Enter your age: 20

You are eligible.

Experiment 10

WAP to explain file handling in Python.

Theory: -

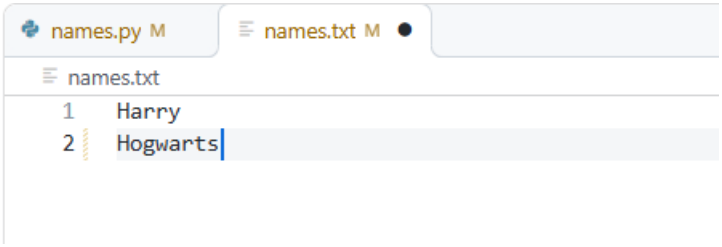
File handling in Python is used to create, read, write, and modify files. Python provides functions like open(), read(), write(), and close() to perform file operations. The open() function is used to open a file in different modes such as read (r), write (w), and append (a). File handling helps store and manage data permanently.

Code: -

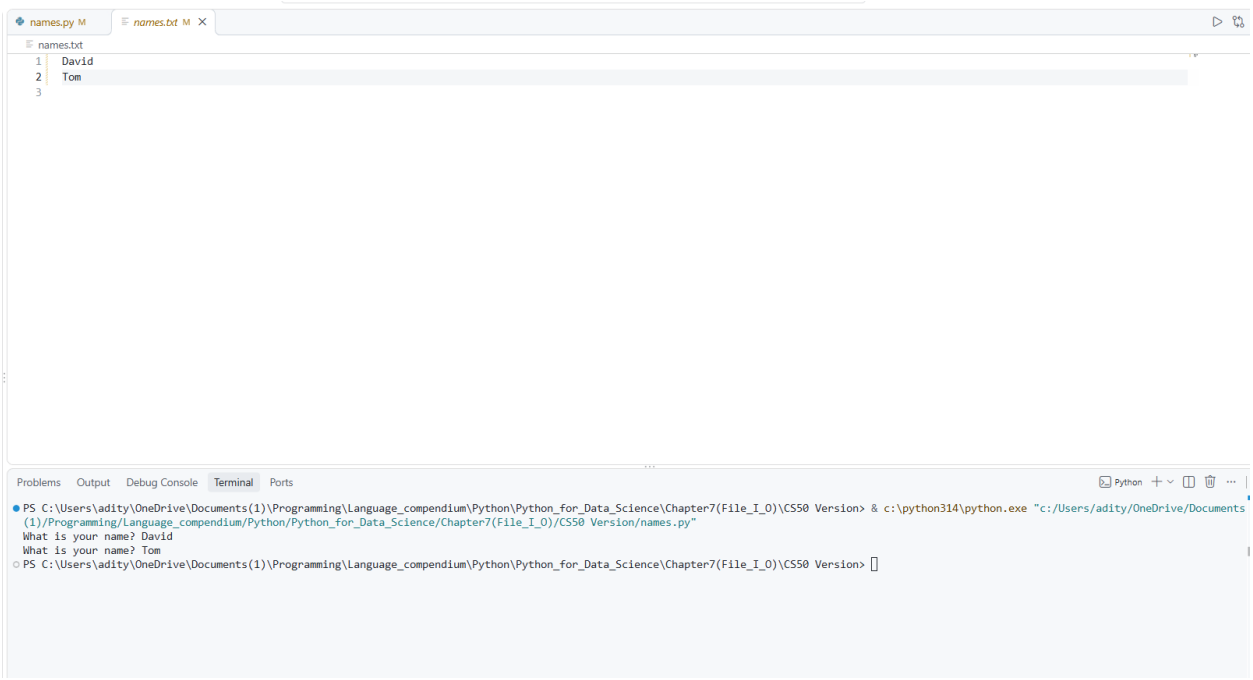
names.py > ...

```
1   name = input('What is your name? ')
2
3   file = open("names.txt", "w") # w (write) #open
4   file.write(f'{name}\n')
5   file.close()# closing
6
7
8   name = input('What is your name? ')
9
10  file = open("names.txt", "a") # a (append)
11  file.write(f'{name}\n')
12  file.close()# closing
13
14
15
```

File before Code Execution: -



File after Code Execution: -



Experiment 11

WAP to demonstrate objects and classes in Python.

Theory: -

A class in Python is a blueprint used to create objects. An object is an instance of a class that contains data and functions. Classes are defined using the class keyword, and objects are created by calling the class name. Classes and objects help make programs organized, reusable, and easy to understand.

Code: -

```
class car: # class
    def __init__(self,brand,model,year):
        self.brand = brand
        self.model  = model
        self.year = year
        self.odometer_reading  = 0
    def print_name(self):
        print(f"{self.brand}  {self.model}  {self.year}")
    def read_odometer(self):
        print(f"This  car has {self.odometer_reading}  kms on it")
    def update_odometer(self,reading):
        self.odometer_reading  = reading
    def travel(self,  distance_travelled):
        self.odometer_reading  += distance_travelled
    def mileage(self,  fuel_used):
        print(f"Mileage  is {(self.odometer_reading/fuel_used):.2f}  km/l")


my_car = car('Audi','A8',2010)  # Objects
```

```
my_car.print_name()  # Calling methods  using object

my_car.update_odometer(2000)

my_car.read_odometer()

my_car.travel(123)

my_car.read_odometer()

my_car.mileage(145)
```

Output: -

```
Audi A8 2010

This car has 2000 kms on it

This car has 2123 kms on it

Mileage is 14.64 km/l
```